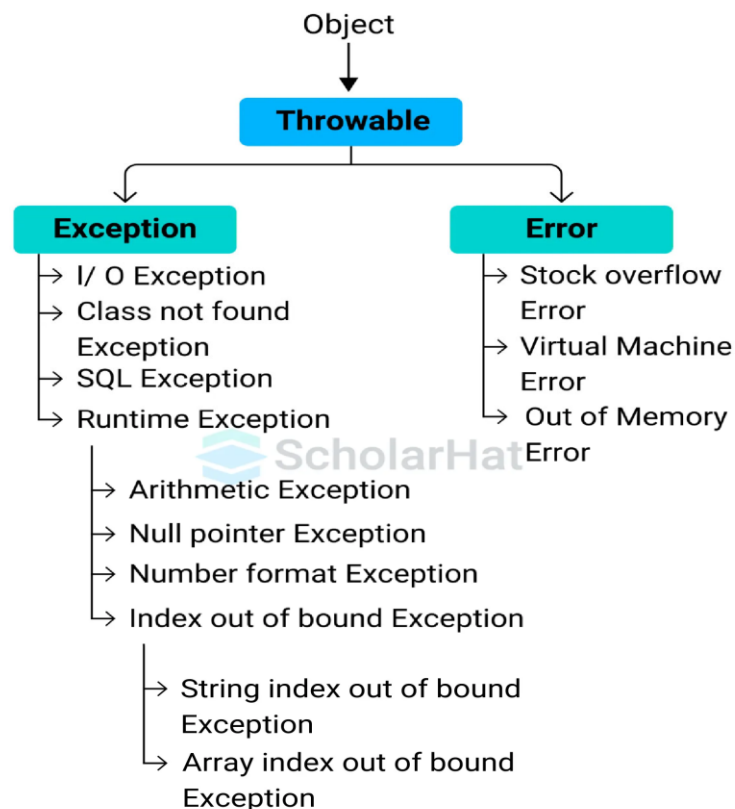


Topic-1: Exception Handling in JAVA

Exception: In java, exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Exception class Hierarchy

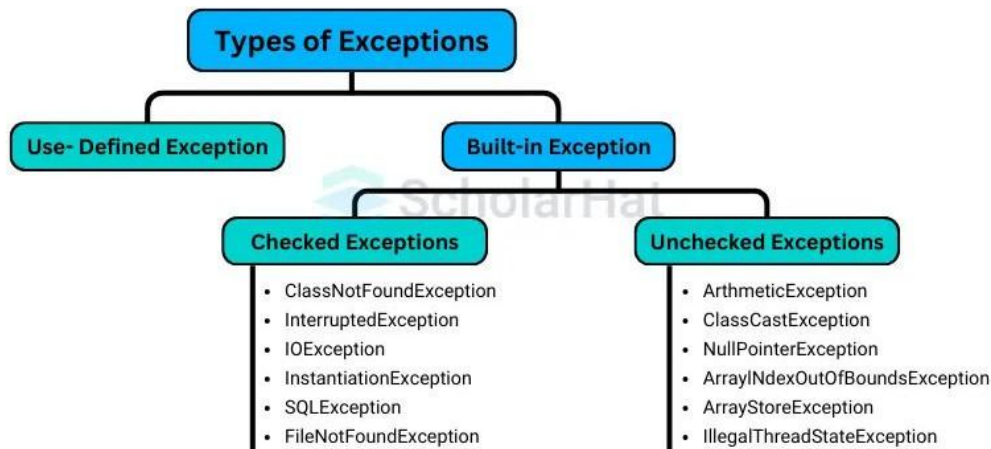
All exception types are subclasses of class **Throwable**, which is at the top of exception class hierarchy.



The hierarchy is divided into two branches:

1. **Errors:** An error is a severe condition that can occur only at run time and is irrecoverable. It prevents a program from executing and cannot be handled by the programmer. An error belongs to the `java.lang.Error` class.
2. **Exceptions:** A programmer can catch and handle exceptions in his program code. When an exception occurs within a method, an object called the Exception object is created. This object contains information about the exception, such as its name and description and the state of the program when the exception occurred.

Exception are categorized into 3 category.



1. Checked Exception

Checked exceptions are classes that inherit directly from the Throwable class, with the exception of RuntimeException and Error. Examples include IOException, SQLException, and so on. Checked exceptions are checked at compilation time. They must be either caught by the code or declared in the method signature using the **throws** keyword.

2. Unchecked Exception

Classes that inherit the RuntimeException class are known as unchecked exceptions. Examples include ArithmeticException, NullPointerException, and ArrayIndexOutOfBoundsException. Unchecked exceptions are not checked at compile-time but rather at runtime by JVM. They do not need to be explicitly caught or declared.

2. User-Defined Exceptions

User-defined exceptions are also known as custom exceptions derived from the **Exception** class from java.lang package. The user creates these exceptions according to different situations. Such exceptions are handled using five keywords: try, catch, throw, throws, and finally.

Errors Vs. Exceptions in Java

Errors

Belongs to the java.lang.Error class

Errors are of Unchecked type

Errors mainly occur during run-time.

Errors are irrecoverable

Exceptions

defined in java.lang.Exception package

Exceptions can be both checked and unchecked.

Only the unchecked exceptions are encountered in run-time.

Exceptions can be handled using exception-handling mechanisms

Topic-2 Exception Handling Technique

Exception Handling

- Exception Handling is the mechanism to handle runtime malfunctions.
- We need to handle such exceptions to prevent abrupt termination of program.
- The term exception means exceptional condition, it is a problem that may arise during the execution of program.
- A bunch of things can lead to exceptions, including programmer error, hardware failures, files that need to be opened cannot be found, resource exhaustion etc.
- Exception handling can be done using *try*, *catch* and *finally* blocks.

a) try block

- try block is used to enclose the code that might throw an exception.
- It must be used within the method.

Syntax:

```
try
{
    //code that may throw exception
}
```

b) catch block

- catch block is used to handle the Exception.
- It must be used after the try block only.
- You can use multiple catch block with a single try.

Syntax:

```
catch(Exception_class_Name ref)
{
}
```

c) finally block

- finally block is optional and can be used only with try-catch block.
- Since exception halts the process of execution, we might have some resources open that will not get closed, so we can use finally block.
- finally block gets executed always, whether exception occurred or not.

Syntax:

```
finally
{
}
```

Example for exception handling technique

```
public class ExceptionDemo {
    public static void main (String[] args) {
        int a=10;
        try{
            for(int i=3;i>=0;i--)
                System.out.println(a/i);
        }//try
        catch(ArithmeticException e){
            System.out.println(e);
        }//catch
        finally {
            // This block will always execute
            System.out.println("This block is executed
regardless of an exception.");
        }//finally

        System.out.println("Program continues after the try-
catch-finally block.");
    }//main
}//class
```

Output:

```
3
5
10
java.lang.ArithmeticException: / by zero
This block is executed regardless of an exception.
Program continues after the try-catch-finally block.
```

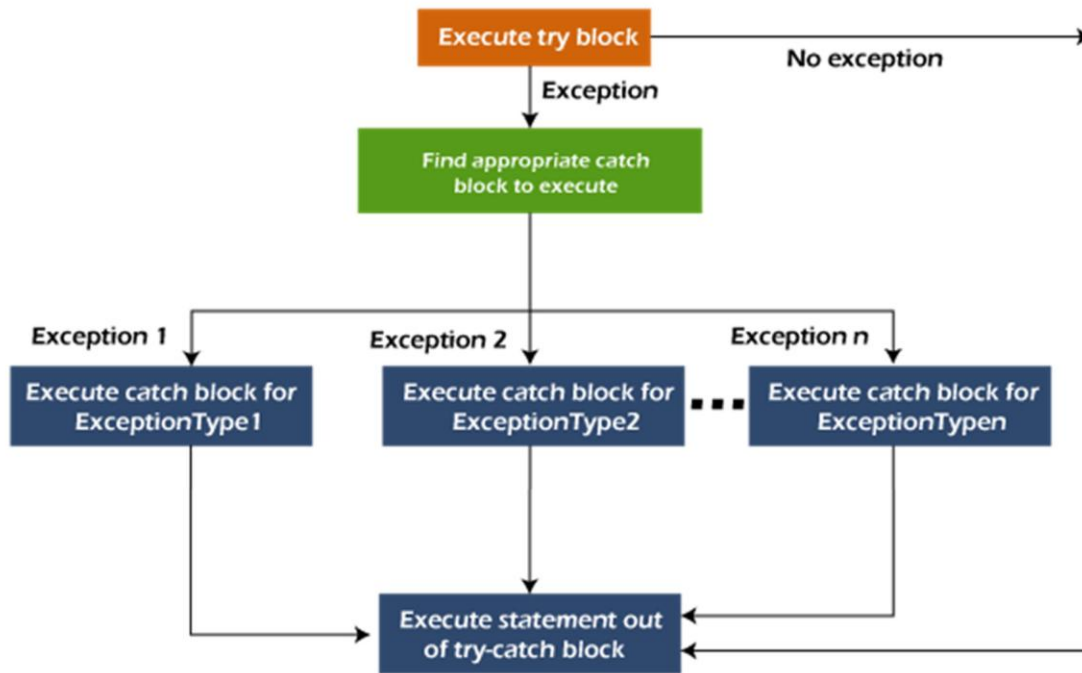
Topic-3: Java Multiple catch blocks

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Points to remember

- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception.

Flowchart of Multi-catch Block

**Example:**

```
class Main {  
    public static void main(String[] args) {  
        try {  
            int array[] = new int[10];  
            array[10] = 30 / 0;  
        } catch (ArithmeticException e) {  
            System.out.println(e.getMessage());  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Output

```
/ by zero
```

Topic-4: 'throws' keyword

- Checked exceptions in Java are exceptions that are checked at compile-time.
- These exceptions must either be caught using a try-catch block or declared in the method signature using the throws keyword.
- Common examples of checked exceptions include IOException, SQLException, and ClassNotFoundException.

Syntax:

```
accessModifier returnType methodName() throws
ExceptionType1, ExceptionType2 ... {
    // code
}
```

Example for throws keyword

```
import java.io.*;
class Main {
    public static void findFile() throws IOException {
        // code that may produce IOException
        File newFile=new File("test.txt");
        FileInputStream stream=new FileInputStream(newFile);
    }

    public static void main(String[] args) {
        try{
            findFile();
        }
        catch(IOException e){
            System.out.println(e);
        }
        System.out.println("Program continues after the exception
handling.");
    }
}
```

Output

```
java.io.FileNotFoundException: test.txt (No such file or directory)
Program continues after the exception handling
```

Topic-5: 'throw' keyword

- "throw" keyword is used to explicitly throw an exception.
- We can throw either checked or unchecked exception in java by throw keyword.
- Only object of Throwable class or its sub classes can be thrown.
- Program execution stops on encountering throw statement, and the closest catch statement is checked for matching type of exception.

Syntax:

```
throw new ThrowableInstance;
```

Example for throw keyword

```
class Test
{
    static void avg()
    {
        try
        {
            throw new ArithmeticException("demo");
        }
        catch(ArithmeticException e)
        {
            System.out.println("Exception caught");
        }
    }
    public static void main(String args[])
    {
        avg();
    }
}
```

Output: Exception caught::java.lang.ArithmeticException: demo.

Difference between throw and throws in Java

throw	throws
Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
Throw is followed by an instance.	Throws is followed by class.
Throw is used within the method.	Throws is used with the method signature.
You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException

Topic-6: User-defined Exception in Java

- If you are creating your own Exception that is known as custom exception or user-defined exception
- By the help of custom exception, you can have your own exception and message.

Steps for creating a user-defined Exception

Step-1 Create your own exception by creating a sub class simply by extending Exception class.

Step-2 You can define a constructor for your Exception sub class (not compulsory) and you can override the toString() function to display your customized message on catch.

Example for User-defined Exception

```
class InvalidAgeException extends Exception{
    InvalidAgeException(String s){
        super(s);
    }
}

class TestCustomException1{

    static void validate(int age)throws InvalidAgeException{
        if(age<18)
            throw new InvalidAgeException("not valid");
        else
            System.out.println("welcome to vote");
    }

    public static void main(String args[]){
        try{
            validate(13);
        }catch(Exception m){System.out.println("Exception
occured: "+m);}

        System.out.println("rest of the code...");
    }
}
```

Output:

```
Exception occured: InvalidAgeException:not valid
rest of the code...
```

Topic-6: Throwable class in java

- The Throwable class is the superclass of all errors and exceptions in the Java.
- Only objects that are instances of this class (or one of its subclasses) are thrown by the Java Virtual Machine or can be thrown by the Java throw statement.
- Similarly, only this class or one of its subclasses can be the argument type in a catch clause.

Example:

```
// Custom Exception Class
class MyCustomException extends Exception {
    public MyCustomException(String message) {
        super(message);
    }
}

// Class to demonstrate the use of the custom exception
class ThrowableExample {

    public static void riskyMethod() throws MyCustomException {
        // Simulating a condition that causes an exception
        boolean condition = true; // Change this to false to avoid
exception
        if (condition) {
            throw new MyCustomException("Something went wrong in
riskyMethod!");
        }
    }

    public static void main(String[] args) {
        try {
            riskyMethod();
        } catch (MyCustomException e) {
            System.out.println("Caught exception: " +
e.getMessage());
            e.printStackTrace(); // Prints the stack trace of the
exception
        }
    }
}
```

Output:

```
Caught exception: Something went wrong in riskyMethod!
MyCustomException: Something went wrong in riskyMethod!
    at ThrowableExample.riskyMethod(ThrowableExample.java:15)
    at ThrowableExample.main(ThrowableExample.java:21)
```

Topic-7: Difference between Checked and Unchecked Exceptions

Checked Exception

Checked exceptions occur at compile time.

The compiler checks a checked exception.

These types of exceptions can be handled at the time of compilation.

They are the sub-class of the exception class.

Here, the JVM needs the exception to catch and handle.

Examples of Checked exceptions:

- FileNotFoundException
- NoSuchFieldException
- InterruptedException
- NoSuchMethodException
- ClassNotFoundException

Unchecked Exception

Unchecked exceptions occur at runtime.

The compiler does not check these types of exceptions.

These types of exceptions cannot be caught or handled at the time of compilation, because they get generated by the mistakes in the program.

They are runtime exceptions and hence are not a part of the Exception class.

Here, the JVM does not require the exception to catch and handle.

Examples of Unchecked Exceptions:

- NoSuchElementException
- UndeclaredThrowableException
- EmptyStackException
- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- SecurityException

Add-on Topic:- Exception propagation

An exception is first thrown from the top of the stack and if it is not caught, it drops down the call stack to the previous method. If not caught there, the exception again drops down to the previous method, and so on until they are caught or until they reach the very bottom of the call stack. This is called exception propagation.

Example for Exception Propagation

```
class TestExceptionPropagation1{  
    void m() {  
        int data=50/0;  
    }  
    void n() {  
        m();  
    }  
    void p() {
```

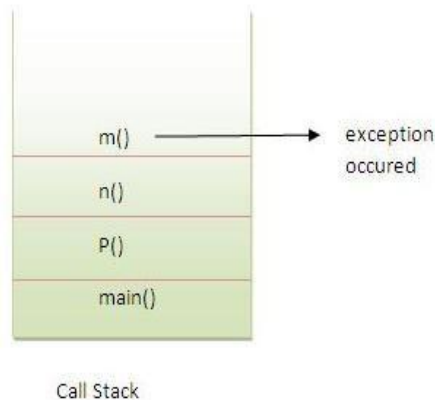
```
try{
    n();
}catch(Exception e){System.out.println("exception handled");
}

}

public static void main(String args[]){
    TestExceptionPropagation1 obj=new TestExceptionPropagation1();
    obj.p();
    System.out.println("normal flow...");
}
}
```

Output:

exception handled
normal flow...



- In the above example exception occurs in `m()` method where it is not handled, so it is propagated to previous `n()` method where it is not handled, again it is propagated to `p()` method where exception is handled.
- Exception can be handled in any method in call stack either in `main()` method, `p()` method, `n()` method or `m()` method.

Add-on Topic: Assertions

- Assertion is a statement in java. It can be used to test your assumptions about the program.
- While executing assertion, it is believed to be true. If it fails, JVM will throw an error named `AssertionError`. It is mainly used for testing purpose.

- It provides an effective way to detect and correct programming errors.

Syntax of using Assertion:

There are two ways to use assertion. First way is:

1. **assert** expression;
2. **assert** expression1 : expression2;

Example of Assertion in java:

```
import java.util.Scanner;
class AssertionExample{
    public static void main( String args[] ){
        Scanner scanner = new Scanner( System.in );
        System.out.print("Enter ur age ");

        int value = scanner.nextInt();
        assert value>=18:" Not valid";
        System.out.println("value is "+value);
    }
}
```

Procedure for Executing the Assertions program

- If you use assertion, It will not run simply because assertion is disabled by default.
- To enable the assertion, **-ea** or **-enableassertions** switch of java must be used.
- Compile it by: **javac AssertionExample.java**
- Run it by: **java -ea AssertionExample**

Output:

Enter ur age 11

Exception in thread "main" java.lang.AssertionError: Not valid